

```

import flet as ft

from datetime import datetime, timedelta

import engine

def generate_analytics_view(page, car_profile):

    view_column = ft.Column(expand=True, scroll=ft.ScrollMode.AUTO, spacing=15)

    current_km = car_profile.get("odometer", {}).get("value", 0)

    tasks = car_profile.get("maintenance_data", {})

    view_column.controls.append(ft.Text("Аналитика износа", size=20, weight=ft.FontWeight.BOLD))

    if not tasks:

        view_column.controls.append(ft.Text("Нет данных", color=ft.Colors.GREY_500, italic=True))

        return view_column

    for t_name, t_data in tasks.items():

        interval = t_data.get("interval", 1)

        remains = (t_data.get("last_service", 0) + interval) - current_km

        passed = max(0, current_km - t_data.get("last_service", 0))

        res = max(0.0, min(1.0, 1.0 - (passed / interval)))

        b_color = ft.Colors.RED_600 if remains <= 0 else (ft.Colors.ORANGE_700 if remains <= 500 else
ft.Colors.GREEN_700)

        st_text = "Срочно заменить!" if remains <= 0 else f"Осталось {remains} км"

        view_column.controls.append(ft.Card(content=ft.Container(content=ft.Column([

            ft.Row([ft.Text(t_name, size=15, weight=ft.FontWeight.BOLD, expand=True), ft.Text(f"{int(res *
100)}%", size=14, color=b_color)], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),

            ft.ProgressBar(value=res, color=b_color, bgcolor=ft.Colors.GREY_200, height=8),

            ft.Text(st_text, size=12, color=ft.Colors.GREY_600, italic=True)

        ], spacing=4), padding=12)))

    return view_column

def show_task_history_dialog(page, db_data, task_name, car_profile, rebuild, show_msg):

```

```

h_col = ft.Column(scroll=ft.ScrollMode.AUTO, height=220, spacing=8)

def refresh():

    h_col.controls.clear()

    t_hist = [h for h in car_profile.get("history", []) if h.get("task") == task_name]

    if not t_hist: h_col.controls.append(ft.Text("Пусто", italic=True))

    else:

        for rec in reversed(t_hist):

            def make_del(r=rec): return lambda _: [car_profile["history"].remove(r),
engine.save_data(db_data), refresh(), rebuild(), show_msg("Удалено")]

            h_col.controls.append(ft.Container(content=ft.Row([

                ft.Column([ft.Row([ft.Text(f"📅 {rec.get('date')}"), ft.Text(f"🚗 {rec.get('odometer')} км")),
ft.Text(rec.get('comment', " ")]),

                ft.IconButton(ft.Icons.DELETE, icon_color=ft.Colors.RED_400, on_click=make_del())

            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN), padding=6, bgcolor=ft.Colors.GREY_50))

        page.update()

    dlg = ft.AlertDialog(title=ft.Text(task_name), content=ft.Container(content=h_col, width=420),
actions=[ft.TextButton("Закрыть", on_click=lambda _: [setattr(dlg, "open", False), page.update()])])

    page.overlay.append(dlg); dlg.open = True; refresh()

def show_add_task_history_dialog(page, db_data, t_name, p_profile, rebuild, show_msg):

    h_odo = ft.TextField(label="Пробег")

    h_date = ft.TextField(label="Дата", value=datetime.now().strftime("%d.%m.%Y"))

    def save(_):

        try:

            km = int(h_odo.value); dt_str = h_date.value.strip(); datetime.strptime(dt_str, "%d.%m.%Y")

            p_profile["history"].append({"task": t_name, "odometer": km, "date": dt_str})

            if km > p_profile["maintenance_data"][t_name]["last_service"]:
p_profile["maintenance_data"][t_name].update({"last_service": km, "date": dt_str})

            engine.save_data(db_data); dlg.open = False; page.update(); rebuild(); show_msg("Добавлено!")

        except: show_msg("Ошибка формата!")

```

```
dlg = ft.AlertDialog(title=ft.Text("Ввод истории"), content=ft.Column([h_odo, h_date], tight=True),
actions=[ft.TextButton("Сохранить", on_click=save)])
```

```
page.overlay.append(dlg); dlg.open = True; page.update()
```

```
def create_task_actions(page, db_data, p, t, current_km, rebuild, show_msg):
```

```
def reset_click(_):
```

```
    now = datetime.now().strftime("%d.%m.%Y")
```

```
    p["maintenance_data"][t].update({"last_service": current_km, "date": now})
```

```
    p["history"].append({"task": t, "odometer": current_km, "date": now})
```

```
    engine.save_data(db_data); rebuild()
```

```
def change_click(_):
```

```
    n_in = ft.TextField(label="Имя", value=t); i_in = ft.TextField(label="Интервал",
value=str(p["maintenance_data"][t]["interval"]))
```

```
def save(_):
```

```
    try:
```

```
        nn = n_in.value.strip(); ni = int(i_in.value)
```

```
        if ni <= 0 or not nn: raise ValueError
```

```
        old = p["maintenance_data"].pop(t); old["interval"] = ni; p["maintenance_data"][nn] = old
```

```
        if nn != t:
```

```
            for h in p.get("history", []):
```

```
                if h["task"] == t: h["task"] = nn
```

```
            engine.save_data(db_data); dlg.open = False; page.update(); rebuild();
show_msg("Изменено")
```

```
        except: show_msg("Ошибка")
```

```
        dlg = ft.AlertDialog(title=ft.Text("Правка"), content=ft.Column([n_in, i_in], tight=True),
actions=[ft.TextButton("OK", on_click=save)])
```

```
        page.overlay.append(dlg); dlg.open = True; page.update()
```

```
def delete_click(_):
```

```
    p["maintenance_data"].pop(t)
```

```
    p["history"] = [h for h in p.get("history", []) if h["task"] != t]
```

```

engine.save_data(db_data); rebuild(); show_msg("Удалено")

return reset_click, change_click, delete_click

def build_maintenance_list(page, db_data, car_name, car_profile, header_card, rebuild, show_msg,
add_task_fn=None):

    c_list = ft.Column(expand=True, scroll=ft.ScrollMode.AUTO); c_list.controls.append(header_card)

    current_km = car_profile.get("odometer", {}).get("value", 0)

    daily = car_profile.get("daily_mileage", 45)

    tasks = car_profile.get("maintenance_data", {})

    if not tasks:

        c_list.controls.append(ft.Container(content=ft.Text("Нет регламентов ТО.",
color=ft.Colors.GREY_500), alignment=ft.Alignment.CENTER, padding=20))

        return c_list

    for t_name, t_data in tasks.items():

        remains = (t_data.get("last_service", 0) + t_data.get("interval", 0)) - current_km

        f_str = (datetime.now() + timedelta(days=remains/daily)).strftime("%d.%m.%Y") if remains > 0 and
daily > 0 else "Срочно ТО!"

        color = ft.Colors.RED_600 if remains <= 0 else (ft.Colors.ORANGE_700 if remains <= 500 else
ft.Colors.GREEN_700)

        sub = f"Осталось: {remains} км | Срок: {f_str}"

        r_fn, c_fn, d_fn = create_task_actions(page, db_data, car_profile, t_name, current_km, rebuild,
show_msg)

        c_list.controls.append(ft.ExpansionTile(title=ft.Text(t_name, weight=ft.FontWeight.BOLD),
subtitle=ft.Text(sub, color=color), controls=[

            ft.Container(content=ft.Column([

                ft.Row([ft.Text(f"Интервал: {t_data.get('interval')} км"), ft.Text(f"Прошлый:
{t_data.get('last_service')} км")),

                ft.Row([

                    ft.Button("История", icon=ft.Icons.HISTORY, on_click=lambda e, tn=t_name:
show_add_task_history_dialog(page, db_data, tn, car_profile, rebuild, show_msg)),

                    ft.IconButton(ft.Icons.EDIT, icon_color=ft.Colors.BLUE_600, on_click=c_fn),

                    ft.IconButton(ft.Icons.CHECK_CIRCLE, icon_color=ft.Colors.GREEN_600, on_click=r_fn),

```

```

        ft.IconButton(ft.Icons.DELETE, icon_color=ft.Colors.RED_400, on_click=d_fn)

    ], alignment=ft.MainAxisAlignment.END)

    )), padding=15, bgcolor=ft.Colors.GREY_50)

    )))

return c_list

def show_car_odometer_history_dialog(page, db_data, car_profile, rebuild, show_msg):

    h_cont = ft.Column(spacing=10, scroll=ft.ScrollMode.AUTO)

    def render():

        h_cont.controls.clear()

        for item in sorted(car_profile.get("odometer_history", []), key=engine.parse_h_date, reverse=True):

            def make_del(i=item): return lambda _: [car_profile["odometer_history"].remove(i),
engine.save_data(db_data), render(), rebuild(), show_msg("Удалено")]

            h_cont.controls.append(ft.Container(content=ft.Row([

                ft.Column([ft.Text(f'{item['value']} км', weight=ft.FontWeight.BOLD), ft.Text(item['date'])]),

                ft.IconButton(ft.Icons.DELETE, on_click=make_del())

            ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN), padding=5, border=ft.Border.all(1,
ft.Colors.BLACK_12)))

        page.update()

    def add_click(_):

        a_km = ft.TextField(label="Пробер"); a_dt = ft.TextField(label="Дата",
value=datetime.now().strftime("%d.%m.%Y"))

        def save(_):

            try:

                v = int(a_km.value); d = a_dt.value.strip(); datetime.strptime(d, "%d.%m.%Y")

                car_profile["odometer_history"].append({"value": v, "date": d})

                if v >= car_profile["odometer"].get("value", 0): car_profile["odometer"] = {"value": v, "date": d}

                car_profile["daily_mileage"] = engine.recalculate_auto_daily_mileage(car_profile)

                engine.save_data(db_data); adlg.open = False; render(); rebuild(); show_msg("Добавлено")

            except: show_msg("Ошибка")

```

```
adlg = ft.AlertDialog(title=ft.Text("Добавить"), content=ft.Column([a_km, a_dt], tight=True),  
actions=[ft.TextButton("OK", on_click=save)])
```

```
page.overlay.append(adlg); adlg.open = True; page.update()
```

```
dlg = ft.AlertDialog(title=ft.Text("История пробы"),  
content=ft.Container(content=ft.Column([ft.Button("✚ Добавить", on_click=add_click), h_cont],  
tight=True), width=400))
```

```
page.overlay.append(dlg); dlg.open = True; render()
```